```
import os
os.environ['PYDEVD_DISABLE_FILE_VALIDATION'] = '1'
```

```
! pip install datasets
```

```
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets) (3.25.2)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.0.2)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: tqdm>=4.66.3 in /usr/local/lib/python3.12/dist-packages (from datasets) (4.67.3)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets
Requirement already satisfied: huggingface-hub>=0.24.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (1.8.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from datasets) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datasets) (6.0.3)
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=2025.3.0,>=2023.1.0->datasets)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (1.4.2)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.28.1)
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (0.24.1)
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub>=0.24.0->datasets) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.4.6)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->datasets) (2026.2.25)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.3)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=20
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2023
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=20
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!=4.0.0a1->fsspec[http]<=2025.3.0,>=2
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0->datasets) (4.13.0)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->huggingface-hub>=0.24.0->datasets) (1.0
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub>=0.24.0->data
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->datasets) (1.17.0)
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->datasets) (8.3.1)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->datasets) (1.5.4)
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->datasets) (13.9.4)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface-hub>=0.24.0->datasets) (0.0.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub>=0.24.0->data
```

```
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer->huggingface-hub>=0.24.0->da
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer->huggingface-hub>=
```

```
from datasets import load_dataset
dataset =  load_dataset("bentrevett/multi30k")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in your Googl
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits
```

```
train,val,test = dataset["train"],dataset["validation"],dataset["test"]
```

Building the tokenization

```
import torch
import torch.nn as nn
import torch.optim as optim
import random
import numpy as np
import spacy
import datasets
import tqdm
```

```
seed = 1234

random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
torch.cuda.manual_seed(seed)
torch.backends.cudnn.deterministic = True
```

```
train[0]
```

```
{'en': 'Two young, White males are outside near many bushes.',
 'de': 'Zwei junge weiße Männer sind im Freien in der Nähe vieler Büsche.'}
```

```
!python -m spacy download en_core_web_sm
!python -m spacy download de_core_news_sm
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any.whl (12.8 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 122.1 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
Collecting de-core-news-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/de_core_news_sm-3.8.0/de_core_news_sm-3.8.0-py3-none-any.whl (14.6 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.6/14.6 MB 109.9 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('de_core_news_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
## loading the tokenizer
en_nlp = spacy.load("en_core_web_sm")
de_nlp = spacy.load("de_core_news_sm")
```

```
sentence = "hi how are you"
[text.text for text in en_nlp.tokenizer(sentence)]
```

```
['hi', 'how', 'are', 'you']
```

```
def tokenize_examples(example,en_nlp,de_nlp,lower,sos_token,eos_token,max_length):
  english = [token.text for token in en_nlp.tokenizer(example["en"])][:max_length]
  german = [token.text  for token in de_nlp.tokenizer(example["de"])][:max_length]
  if lower:
    english = [text.lower() for text in english]
    german = [text.lower() for text in german]
  english = [sos_token] + english + [eos_token]
  german = [sos_token] + german + [eos_token]
  return {"en_tokens":english,"de_tokens":german}
```

```
tokenize_examples(train[0],en_nlp,de_nlp,1,"<sos>","<eos>",100)
```

```
{'en_tokens': ['<sos>',
  'two',
  'young',
```

```
    ',',
    'white',
    'males',
    'are',
    'outside',
    'near',
    'many',
    'bushes',
    '.',
    '<eos>'],
   'de_tokens': ['<sos>',
    'zwei',
    'junge',
    'weiße',
    'männer',
    'sind',
    'im',
    'freien',
    'in',
    'der',
    'nähe',
    'vieler',
    'büsche',
    '.',
    '<eos>']}
```

```
max_length = 128
lower = True
sos_token = "<sos>"
eos_token = "<eos>"

fn_kwargs = {
    "en_nlp": en_nlp,
    "de_nlp": de_nlp,
    "lower": lower,
    "sos_token": sos_token,
    "eos_token": eos_token,
    "max_length": max_length
}
```

```
train_data = train.map(tokenize_examples, fn_kwargs=fn_kwargs)
valid_data = val.map(tokenize_examples, fn_kwargs=fn_kwargs)
test_data = test.map(tokenize_examples, fn_kwargs=fn_kwargs)
```

```
Map: 100%  29000/29000 [00:12<00:00, 3880.36 examples/s]
Map: 100%  1014/1014 [00:00<00:00, 5096.11 examples/s]
Map: 100%  1000/1000 [00:00<00:00, 4901.42 examples/s]
```

Vocabulary

```
## so lets build the vocabulary
unk_token = "<unk>" # to capture the unknown token while inference
pad_token = "<pad>" # to make the len same
special_tokens = [
    unk_token,
    pad_token,
    sos_token,
    eos_token,
]
```

```
train_data["en_tokens"][0]
```

```
['<sos>',
 'two',
 'young',
 ',',
 'white',
 'males',
 'are',
 'outside',
 'near',
 'many',
 'bushes',
 '.',
 '<eos>']
```

```
from collections import Counter
vocab = {
    "<unk>":0,
    "<pad>":1,
    "<sos>":2,
    "<eos>":3
}
```

```
Counter
```

```
collections.Counter
def __init__(self, iterable=None, /, **kwds)
```

```
Dict subclass for counting hashable items.  Sometimes called a bag
or multiset.  Elements are stored as dictionary keys and their counts
are stored as dictionary values.

>>> c = Counter('abcdeabcdabcaba')  # count elements from a string
```

```
for sentence in train_data["en_tokens"]:
  for token in sentence:
    if token not in vocab:
      vocab[token] = len(vocab) + 1
  break
```

```
sentence
```

```
['<sos>',
 'two',
 'young',
 ',',
 'white',
 'males',
 'are',
 'outside',
 'near',
 'many',
 'bushes',
 '.',
 '<eos>']
```

```
vocab
```

```
{'<unk>': 0,
 '<pad>': 1,
 '<sos>': 2,
 '<eos>': 3,
 'two': 5,
 'young': 6,
 ',': 7,
 'white': 8,
 'males': 9,
 'are': 10,
 'outside': 11,
```

```
 'near': 12,
 'many': 13,
 'bushes': 14,
 '.': 15}
```

```
en_vocab =  {
    "<unk>":0,
    "<pad>":1,
    "<sos>":2,
    "<eos>":3
}
# english
en_counter = Counter()
for sentence in train_data["en_tokens"]:
    en_counter.update(sentence)
min_freq = 2
for token, count in en_counter.items():
    if count >= min_freq and token not in en_vocab:
        en_vocab[token] = len(en_vocab)
```

```
en_vocab
```

Show hidden output

```
de_vocab =  {
    "<unk>":0,
    "<pad>":1,
    "<sos>":2,
    "<eos>":3
}
# german
de_counter = Counter()
for sentence in train_data["de_tokens"]:
    de_counter.update(sentence)
min_freq = 2
for token, count in de_counter.items():
    if count >= min_freq and token not in de_vocab:
        de_vocab[token] = len(de_vocab)
```

```
en_vocab["the"]
```

```
41
```

```
len(en_vocab),len(de_vocab)
```

```
(5893, 7853)
```

```
# just and test
"The" in en_vocab
```

```
False
```

```
assert en_vocab[unk_token] == de_vocab[unk_token]
assert en_vocab[pad_token] == de_vocab[pad_token]

unk_index = en_vocab[unk_token]
pad_index = en_vocab[pad_token]
# to check that they got the came index
```

```
unk_index , pad_index
```

```
(0, 1)
```

```
def lookup_indices(example,en_vocab,de_vocab):
  en_indices,de_indices = [],[]
  #english
  for token in example["en_tokens"]:
    if token not in en_vocab:
      en_indices.append(en_vocab["<unk>"])
    else:  # Add this else!
        en_indices.append(en_vocab[token])
  #german
  for token in example["de_tokens"]:
    if token not in de_vocab:
      de_indices.append(de_vocab["<unk>"])
    else:  # Add this else!
        de_indices.append(de_vocab[token])
  return {"en_ids":torch.tensor(en_indices),"de_ids":torch.tensor(de_indices)}
```

```
lookup_indices(train_data[0],en_vocab,de_vocab)
```

```
{'en_ids': tensor([ 2,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14,  3]),
 'de_ids': tensor([ 2,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,  3])}
```

```
train[0]
```

```
{'en': 'Two young, White males are outside near many bushes.',
 'de': 'Zwei junge weiße Männer sind im Freien in der Nähe vieler Büsche.'}
```

```
train_data[0]
```

Show hidden output

```
fn_kwargs = {
    "en_vocab":en_vocab,
    "de_vocab":de_vocab
}
train_data = train_data.map(lookup_indices,fn_kwargs=fn_kwargs)
```

Map: 100% 29000/29000 [00:06<00:00, 5519.26 examples/s]

```
train_data[0]
```

```
{'en': 'Two young, White males are outside near many bushes.',
 'de': 'Zwei junge weiße Männer sind im Freien in der Nähe vieler Büsche.',
 'en_tokens': ['<sos>',
  'two',
  'young',
  ',',
  'white',
  'males',
  'are',
  'outside',
  'near',
  'many',
  'bushes',
  '.',
  '<eos>'],
 'de_tokens': ['<sos>',
  'zwei',
  'junge',
  'weiße',
  'männer',
  'sind',
  'im',
  'freien',
  'in',
  'der',
  'nähe',
  'vieler',
  'büsche',
  '.',
```

```
  '<eos>'],
 'en_ids': [2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 3],
 'de_ids': [2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 3]}
```

```
valid_data = valid_data.map(lookup_indices,fn_kwargs=fn_kwargs)
test_data = test_data.map(lookup_indices,fn_kwargs=fn_kwargs)
```

```
Map: 100%                                          1014/1014 [00:00<00:00, 6898.25 examples/s]

Map: 100%                                          1000/1000 [00:00<00:00, 7304.74 examples/s]
```

```
# reverse vocab
en_itos = {i: s for s, i in en_vocab.items()}
de_itos = {i: s for s, i in de_vocab.items()}

def lookup_vocab(example,en_itos,de_itos):
  en_indices,de_indices = [],[]
  #english
  for ids in example["en_ids"]:
    en_indices.append(en_itos[ids])
  #german
  for ids in example["de_ids"]:
    de_indices.append(de_itos[ids])
  return {"en_tokens_rev":(en_indices),"de_tokens_rev":(de_indices)}
```

```
train_data
```

```
Dataset({
    features: ['en', 'de', 'en_tokens', 'de_tokens', 'en_ids', 'de_ids'],
    num_rows: 29000
})
```

```
lookup_indices(train_data[0],en_vocab,de_vocab)
```

```
{'en_ids': tensor([ 2,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14,  3]),
 'de_ids': tensor([ 2,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,  3])}
```

```
train_data[0]
```

```
{'en': 'Two young, White males are outside near many bushes.',
 'de': 'Zwei junge weiße Männer sind im Freien in der Nähe vieler Büsche.',
 'en_tokens': ['<sos>',
  'two',
  'young',
```

```
      ',',
      'white',
      'males',
      'are',
      'outside',
      'near',
      'many',
      'bushes',
      '.',
      '<eos>'],
     'de_tokens': ['<sos>',
      'zwei',
      'junge',
      'weiße',
      'männer',
      'sind',
      'im',
      'freien',
      'in',
      'der',
      'nähe',
      'vieler',
      'büsche',
      '.',
      '<eos>'],
     'en_ids': [2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 3],
     'de_ids': [2, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 3]}
```

Data Loader

```
def get_collate_fn(pad_index):
    def collate_fn(batch):
        batch_en_ids = [torch.tensor(example["en_ids"]) for example in batch]
        batch_de_ids = [torch.tensor(example["de_ids"]) for example in batch]
        batch_en_ids = nn.utils.rnn.pad_sequence(batch_en_ids, padding_value=pad_index) # padding as the same length for one batch
        batch_de_ids = nn.utils.rnn.pad_sequence(batch_de_ids, padding_value=pad_index)
        batch = {
            "en_ids": batch_en_ids,
            "de_ids": batch_de_ids,
        }
        return batch

    return collate_fn
```

```
def get_data_loader(dataset, batch_size, pad_index, shuffle=False):
    collate_fn = get_collate_fn(pad_index)
    data_loader = torch.utils.data.DataLoader(
        dataset=dataset,
        batch_size=batch_size,
        collate_fn=collate_fn,
        shuffle=shuffle,
    )
    return data_loader
```

```
batch_size = 32

train_data_loader = get_data_loader(train_data, batch_size, pad_index, shuffle=True)
valid_data_loader = get_data_loader(valid_data, batch_size, pad_index)
test_data_loader = get_data_loader(test_data, batch_size, pad_index)
```

```
for x in train_data_loader:
  break
```

```
len(x["en_ids"]) , len(x["de_ids"])
```

```
(26, 24)
```

Next model training

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import math
```

```
## Embedding
class inputEmbedding(nn.Module):
  def __init__(self,vocab_size,embedding_dim):
    super().__init__()
    self.vocab_size = vocab_size
    self.embedding_dim = embedding_dim
    self.embedding = nn.Embedding(vocab_size,embedding_dim)
  def forward(self,x):
    x = self.embedding(x)
    return x
```

```
## Positional Encoding
class positionalEncoding(nn.Module):
  def __init__(self,seq_len,d_k):
    super().__init__()
    self.seq_len = seq_len
    self.d_k = d_k
    pe = torch.zeros(seq_len,d_k)
    div = torch.exp(torch.arange(0,self.d_k,2).float() * - (math.log(10000.0)/self.d_k))
    positions = torch.arange(0,self.seq_len,dtype=torch.float).unsqueeze(1)
    pe[:,0::2] = torch.sin(positions*div)
    pe[:,1::2] = torch.cos(positions*div)
    pe = pe.unsqueeze(0)
    self.register_buffer("pe",pe)
  def forward(self,x):
    return x + self.pe[:,:x.size(1),:].requires_grad_(False)
```

```
## Feed Forward Network
class FeedForward(nn.Module):
  def __init__(self,d_k):
    super().__init__()
    self.d_k = d_k
    self.fnn = nn.Sequential(
        nn.Linear(self.d_k,2048),
        nn.ReLU(),
        nn.Linear(2048,self.d_k)
    )
  def forward(self,x):
    return self.fnn(x)
```

```
## layer norm
class layerNorm(nn.Module):
  def __init__(self,embedding_dim):
    super().__init__()
    self.alpha = nn.Parameter(torch.ones(embedding_dim))
    self.beta = nn.Parameter(torch.ones(embedding_dim))
    self.eps = 0.001
    pass
  def forward(self,x):
    mean = x.mean(dim=-1, keepdim=True)
    std = x.std(dim=-1, keepdim=True)
```

```
    x = self.alpha * (x-mean)/(self.eps+std) + self.beta
    return x
```

```
## Multihead Self Attention
class MultiHeadAttention(nn.Module):
  def __init__(self,d_k,heads,head_dim,device):
    super().__init__()
    self.d_k = d_k
    assert d_k == heads*head_dim
    self.heads = heads
    self.head_dim = head_dim
    self.Q = nn.Linear(self.d_k,self.d_k)
    self.K = nn.Linear(self.d_k,self.d_k)
    self.V = nn.Linear(self.d_k,self.d_k)
    self.device = device
    self.out = nn.Linear(self.d_k,self.d_k)
  def forward(self,x,mask):
    B,S,E = x.size()
    q = self.Q(x).view(B,S,self.heads,self.head_dim).transpose(1,2) # [32,100,128] => [32,100,4,32]
    k = self.K(x).view(B,S,self.heads,self.head_dim).transpose(1,2)
    v = self.V(x).view(B,S,self.heads,self.head_dim).transpose(1,2)
    k = k.transpose(-2,-1) # [32,4,100,32]
    attention_scores = ((q @ k)/math.sqrt(self.d_k))
    if mask :
      mat = torch.tril(torch.ones(S,S)).to(self.device)
      mat = mat.unsqueeze(0).unsqueeze(0)
      attention_scores = attention_scores.masked_fill(mat == 0,-1e9) ## where where their is 0 put to -infinity
    attention_scores = torch.softmax(attention_scores,dim=-1) @ v
    attention_score = attention_scores.transpose(1,2).contiguous().view(B,S,E)
    return self.out(attention_score)
```

```
## cross attention
class CrossAttention(nn.Module):
  def __init__(self,d_k,heads,head_dim):
    super().__init__()
    self.d_k = d_k
    assert d_k == heads*head_dim
    self.heads = heads
    self.head_dim = head_dim
    self.Q = nn.Linear(self.d_k,self.d_k)
    self.out = nn.Linear(self.d_k,self.d_k)
  def forward(self,x,k,v):
    B,S,E = x.size()
    B, S_enc, E_enc = k.size()
    q = self.Q(x).view(B,S,self.heads,self.head_dim).transpose(1,2)
```

```
    k = k.view(B,S_enc,self.heads,self.head_dim).transpose(1,2)
    v = v.view(B,S_enc,self.heads,self.head_dim).transpose(1,2)
    k = k.transpose(-2,-1) # [32,4,100,32]
    attention_scores = ((q @ k)/math.sqrt(self.d_k))
    attention_scores = torch.softmax(attention_scores,dim=-1) @ v
    attention_score = attention_scores.transpose(1,2).contiguous().view(B,S,E)
    return self.out(attention_score)
```

```
## residual layer
class residualLayer(nn.Module):
  def __init__(self):
    super().__init__()
    self.norm = layerNorm()
    self.dropout = nn.Dropout(0.1)
  def forward(self,x,sub_layer):
    return self.norm(x + self.dropout(sub_layer))
```

```
class EncoderBlock(nn.Module):
  def __init__(self,attention:MultiHeadAttention,fnn:FeedForward,d_k):
    super().__init__()
    self.attention = attention
    self.attention = attention
    self.fnn = fnn
    self.norm = layerNorm(d_k)
  def forward(self,x):
    # here the x coems is positionally encoded
    x = self.norm(x + self.attention(x,0))
    x = self.norm(x + self.fnn(x))
    return x
```

```
class DecoderBlock(nn.Module):
  def __init__(self,attention:MultiHeadAttention,fnn:FeedForward,cr:CrossAttention,d_k):
    super().__init__()
    self.attention = attention
    self.attention = attention
    self.fnn = fnn
    self.norm = layerNorm(d_k)
    self.cr = cr
    # self.k = nn.Linear(d_k,d_k)
    # self.V = nn.Linear(d_k,d_k)
  def forward(self,x,k,v):
    # here the x coems is positionally encoded
    x = self.norm(x + self.attention(x,1))
    x = self.norm(x + self.cr(x,k,v))
```

```
    x = self.norm(x + self.fnn(x))
    return x
```

```
class InitialLayer(nn.Module):
  def __init__(self,embedding:inputEmbedding,positional:positionalEncoding):
    super().__init__()
    self.embedding = embedding
    self.pos = positional
  def forward(self,x):
    x = self.pos(self.embedding(x))
    return x
```

```
class OutputLayer(nn.Module):
  def __init__(self,d_k,vocab_size):
    super().__init__()
    self.d_k = d_k
    self.out = nn.Linear(d_k,vocab_size)
  def forward(self,x):
    return self.out(x) # cross entropy as softmax
```

```
from torch.nn.modules import ModuleList
class Encoder(nn.Module):
  def __init__(self,sublist:nn.ModuleList,d_k):
    super().__init__()
    self.sublayer = sublist
    self.norm = layerNorm(d_k)
  def forward(self,x):
    for layer in self.sublayer:
      x = layer(x)
    return self.norm(x)
```

```
from torch.nn.modules import ModuleList
class Decoder(nn.Module):
  def __init__(self,sublist:nn.ModuleList,d_k):
    super().__init__()
    self.sublayer = sublist
    self.norm = layerNorm(d_k)
  def forward(self,x,k,v):
    for layer in self.sublayer:
      x = layer(x,k,v)
    return self.norm(x)
```

```
class Transformer(nn.Module):
  def __init__(self,encoder:Encoder,decoder:Decoder,initial:InitialLayer,out:OutputLayer,d_k):
    super().__init__()
    self.encoder = encoder
    self.decoder = decoder
    self.out = out
    self.initial = initial
    self.K = nn.Linear(d_k,d_k)
    self.V = nn.Linear(d_k,d_k)
  def forward(self,x,y):
    x = self.initial(x)
    y = self.initial(y)
    x = self.encoder(x)
    k,v = self.K(x),self.V(x)
    #print(k.shape , v.shape)
    y = self.decoder(y,k,v)
    y = self.out(y)
    return y
```

```
# --- Parameters ---
d_k = 128
heads = 4
head_dim = 32
v_size = len(de_vocab)
seq_len = max_length
device = "cuda" if torch.cuda.is_available() else "cpu"
# 1. Basic Components
emb = inputEmbedding(v_size, d_k)
pos = positionalEncoding(seq_len, d_k)
it = InitialLayer(emb, pos)
fnn = FeedForward(d_k)
att = MultiHeadAttention(d_k, heads, head_dim, device)
cr = CrossAttention(d_k, heads, head_dim)
out_layer = OutputLayer(d_k, v_size)

# 2. Build Stacks using ModuleList
# We create 6 unique layers for Encoder and Decoder
enc_blocks = nn.ModuleList([EncoderBlock(att, fnn, d_k) for _ in range(6)])
dec_blocks = nn.ModuleList([DecoderBlock(att, fnn, cr, d_k) for _ in range(6)])

encoder_stack = Encoder(enc_blocks, d_k)
decoder_stack = Decoder(dec_blocks, d_k)

# 3. Initialize Transformer
```

```
model = Transformer(encoder_stack, decoder_stack, it, out_layer, d_k).to(device)
# Expected: torch.Size([32, 1000, 5000])
```

```
len(en_vocab) , len(de_vocab)
```

```
(5893, 7853)
```

Training transformer model for machine translation task

```
import torch.optim as optim
from torch.optim import lr_scheduler as schedular
```

test part

```
optimizer = optim.Adam(model.parameters(), lr=0.0001)
criterion = nn.CrossEntropyLoss(ignore_index=pad_index)
num_epochs = 1
for epoch in range(num_epochs):
    model.train()
    train_loss = 0

    for batch in train_data_loader:
        src = batch["en_ids"].to(device).transpose(0, 1) # Ensure [Batch, Seq] i.e [26,32] => [32,26]
        trg = batch["de_ids"].to(device).transpose(0, 1)

        optimizer.zero_grad()
        #print(len(src) , len(trg[:,:-1]))
        # Output: [batch, trg_len, vocab_size]
        output = model(src, trg[:, :-1]) # $[32, 23]$We remove the <eos> (last token) so the model predicts it.

        # Flatten for CrossEntropy
        output_dim = output.shape[-1] # the cross entropy expect the pred as the 2d matrix
        output = output.contiguous().view(-1, output_dim)
        trg = trg[:, 1:].contiguous().view(-1) # the cross entropy except the target as the 1d matrix

        loss = criterion(output, trg)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        train_loss += loss.item()
```

Show hidden output

```
epoch_loss / len(train_data_loader)
```

```
4.913483351567614
```

test part over

```
import math
import time
from tqdm.notebook import tqdm

def evaluate(model, iterator, criterion):
    model.eval()
    epoch_loss = 0
```

```
    # Wrap validation loader with tqdm
    val_pbar = tqdm(iterator, desc="Evaluating", leave=False)

    with torch.no_grad():
        for batch in val_pbar:
            src = batch["en_ids"].to(device).transpose(0, 1)
            trg = batch["de_ids"].to(device).transpose(0, 1)

            output = model(src, trg[:, :-1])

            output_dim = output.shape[-1]
            output = output.contiguous().view(-1, output_dim)
            trg = trg[:, 1:].contiguous().view(-1)

            loss = criterion(output, trg)
            epoch_loss += loss.item()

            # Optional: Update progress bar with loss
            val_pbar.set_postfix(loss=f"{loss.item():.4f}")

    return epoch_loss / len(iterator)

# Training Hyperparameters
num_epochs = 10
best_valid_loss = float('inf')

for epoch in range(num_epochs):
    start_time = time.time()

    # --- Training Phase ---
    model.train()
    train_loss = 0

    # Wrap training loader with tqdm
    train_pbar = tqdm(train_data_loader, desc=f"Epoch {epoch+1}/{num_epochs}")

    for batch in train_pbar:
        src = batch["en_ids"].to(device).transpose(0, 1)
        trg = batch["de_ids"].to(device).transpose(0, 1)

        optimizer.zero_grad()
        output = model(src, trg[:, :-1])

        output_dim = output.shape[-1]
        output = output.contiguous().view(-1, output_dim)
        trg = trg[:, 1:].contiguous().view(-1)
```

```
        loss = criterion(output, trg)
        loss.backward()

        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        train_loss += loss.item()

        # Update progress bar with current batch loss
        train_pbar.set_postfix(loss=f"{loss.item():.4f}")

    # --- Validation Phase ---
    avg_train_loss = train_loss / len(train_data_loader)
    avg_valid_loss = evaluate(model, valid_data_loader, criterion)

    end_time = time.time()

    # --- Checkpointing ---
    if avg_valid_loss < best_valid_loss:
        best_valid_loss = avg_valid_loss
        torch.save(model.state_dict(), 'transformer_best_model_epoch=10.pt')
        print(f"✔ Epoch {epoch+1}: New best model saved (Val Loss: {avg_valid_loss:.3f})")

    print(f'Epoch: {epoch+1:02} | Time: {end_time - start_time:.2f}s')
    print(f'\tTrain Loss: {avg_train_loss:.3f} | Train PPL: {math.exp(avg_train_loss):7.3f}')
    print(f'\t Val. Loss: {avg_valid_loss:.3f} |  Val. PPL: {math.exp(avg_valid_loss):7.3f}')
```

```
Epoch 1/10: 100%                                        907/907 [00:56<00:00, 16.06it/s, loss=2.9591]


✔ Epoch 1: New best model saved (Val Loss: 3.257)
Epoch: 01 | Time: 56.64s
    Train Loss: 3.452 | Train PPL:  31.562
     Val. Loss: 3.257 |  Val. PPL:  25.971
Epoch 2/10: 100%                                        907/907 [00:58<00:00, 16.01it/s, loss=2.6411]


✔ Epoch 2: New best model saved (Val Loss: 3.039)
Epoch: 02 | Time: 58.95s
    Train Loss: 3.186 | Train PPL:  24.194
     Val. Loss: 3.039 |  Val. PPL:  20.884
Epoch 3/10: 100%                                        907/907 [01:00<00:00, 15.87it/s, loss=2.8216]


✔ Epoch 3: New best model saved (Val Loss: 2.863)
Epoch: 03 | Time: 60.94s
    Train Loss: 2.969 | Train PPL:  19.474
     Val. Loss: 2.863 |  Val. PPL:  17.515
Epoch 4/10: 100%                                        907/907 [00:58<00:00, 16.42it/s, loss=2.3300]
```

```
torch.cuda.empty_cache()
```

```
    Train Loss: 2.784 | Train PPL:  16.178
```

```
import math
import time
from tqdm.notebook import tqdm

def evaluate(model, iterator, criterion):
    model.eval()
    epoch_loss = 0
    val_pbar = tqdm(iterator, desc="Evaluating", leave=False)

    with torch.no_grad():
        for batch in val_pbar:
            src = batch["en_ids"].to(device).transpose(0, 1)
            trg = batch["de_ids"].to(device).transpose(0, 1)

            output = model(src, trg[:, :-1])

            output_dim = output.shape[-1]
            output = output.contiguous().view(-1, output_dim)
```

```
            trg = trg[:, 1:].contiguous().view(-1)

            loss = criterion(output, trg)
            epoch_loss += loss.item()
            val_pbar.set_postfix(loss=f"{loss.item():.4f}")

    return epoch_loss / len(iterator)

# --- Hyperparameters & Early Stopping Setup ---
num_epochs = 50  # Increased since Early Stopping will handle the cutoff
best_valid_loss = float('inf')
patience = 7      # How many epochs to wait for improvement before stopping
counter = 0       # Tracks epochs without improvement

for epoch in range(num_epochs):
    start_time = time.time()

    # --- Training Phase ---
    model.train()
    train_loss = 0
    train_pbar = tqdm(train_data_loader, desc=f"Epoch {epoch+1}/{num_epochs}")

    for batch in train_pbar:
        src = batch["en_ids"].to(device).transpose(0, 1)
        trg = batch["de_ids"].to(device).transpose(0, 1)

        optimizer.zero_grad()
        output = model(src, trg[:, :-1])

        output_dim = output.shape[-1]
        output = output.contiguous().view(-1, output_dim)
        trg = trg[:, 1:].contiguous().view(-1)

        loss = criterion(output, trg)
        loss.backward()

        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        train_loss += loss.item()
        train_pbar.set_postfix(loss=f"{loss.item():.4f}")

    # --- Validation Phase ---
    avg_train_loss = train_loss / len(train_data_loader)
    avg_valid_loss = evaluate(model, valid_data_loader, criterion)
```

```
        end_time = time.time()

        # --- Checkpointing & Early Stopping Logic ---
        if avg_valid_loss < best_valid_loss:
            best_valid_loss = avg_valid_loss
            torch.save(model.state_dict(), 'transformer_best_model_epoch=50.pt')
            print(f"✅ Epoch {epoch+1}: New best model saved! (Val Loss: {avg_valid_loss:.3f})")
            counter = 0  # Reset counter because we found a better model
        else:
            counter += 1
            print(f"⚠️ No improvement in Val Loss for {counter} epoch(s).")

        print(f'Epoch: {epoch+1:02} | Time: {end_time - start_time:.2f}s')
        print(f'\tTrain Loss: {avg_train_loss:.3f} | Train PPL: {math.exp(avg_train_loss):7.3f}')
        print(f'\t Val. Loss: {avg_valid_loss:.3f} |  Val. PPL: {math.exp(avg_valid_loss):7.3f}')

        if counter >= patience:
            print(f"🛑 Early stopping triggered. Training finished at Epoch {epoch+1}.")
            break
```

```
Epoch 1/50: 100%                                        907/907 [01:03<00:00, 15.90it/s, loss=1.2847]


✅ Epoch 1: New best model saved! (Val Loss: 1.876)
Epoch: 01 | Time: 64.43s
    Train Loss: 1.029 | Train PPL:   2.798
     Val. Loss: 1.876 |  Val. PPL:   6.525
Epoch 2/50: 100%                                        907/907 [01:04<00:00, 14.53it/s, loss=0.7204]


⚠️ No improvement in Val Loss for 1 epoch(s).
Epoch: 02 | Time: 65.72s
    Train Loss: 0.993 | Train PPL:   2.699
     Val. Loss: 1.894 |  Val. PPL:   6.643
Epoch 3/50: 100%                                        907/907 [01:06<00:00, 13.10it/s, loss=1.0136]


⚠️ No improvement in Val Loss for 2 epoch(s).
Epoch: 03 | Time: 67.42s
    Train Loss: 0.959 | Train PPL:   2.609
     Val. Loss: 1.896 |  Val. PPL:   6.659
Epoch 4/50: 100%                                        907/907 [00:57<00:00, 16.85it/s, loss=0.7464]


⚠️ No improvement in Val Loss for 3 epoch(s).
Epoch: 04 | Time: 58.72s
    Train Loss: 0.926 | Train PPL:   2.524
     Val. Loss: 1.904 |  Val. PPL:   6.711
Epoch 5/50: 100%                                        907/907 [00:59<00:00, 13.92it/s, loss=0.9689]


⚠️ No improvement in Val Loss for 4 epoch(s).
Epoch: 05 | Time: 60.72s
    Train Loss: 0.894 | Train PPL:   2.444
     Val. Loss: 1.917 |  Val. PPL:   6.797
Epoch 6/50: 100%                                        907/907 [01:01<00:00, 14.81it/s, loss=0.8233]


⚠️ No improvement in Val Loss for 5 epoch(s).
Epoch: 06 | Time: 61.91s
    Train Loss: 0.863 | Train PPL:   2.369
     Val. Loss: 1.926 |  Val. PPL:   6.862
Epoch 7/50: 100%                                        907/907 [00:59<00:00, 15.97it/s, loss=1.0403]


⚠️ No improvement in Val Loss for 6 epoch(s).
```

```
Epoch: 07 | Time: 60.27s
	Train Loss: 0.834 | Train PPL:   2.302
```

Inference code

```
def translate_sentence(sentence, model, en_vocab, de_itos, device, max_len=50):
    model.eval()

    # 1. Tokenize and convert to indices
    tokens = [token.text.lower() for token in en_nlp.tokenizer(sentence)]
    tokens = ['<sos>'] + tokens + ['<eos>']
    src_indices = [en_vocab.get(token, en_vocab['<unk>']) for token in tokens]

    # 2. Convert to Tensor and move to device [Batch, Seq]
    src_tensor = torch.LongTensor(src_indices).unsqueeze(0).to(device)

    # 3. Start the decoder with just <sos>
    trg_indices = [de_vocab['<sos>']]

    for i in range(max_len):
        trg_tensor = torch.LongTensor(trg_indices).unsqueeze(0).to(device)

        with torch.no_grad():
            output = model(src_tensor, trg_tensor)
```